PYTHON ACTIVITY-BASED LEARNING

REPORT ON 10 SELECTED CASE STUDIES

Submitted in partial fulfillment of the requirements for the subject Python Programming

Submitted by

Name: Abdul Khadeer Khan

Roll Number: 26000850

Branch: CSE (AI/ML)

Section: 1

Subject: Python Programming

Academic Year: 2026–2027

Project Title: Implementation of Ten Python Case Studies

GitHub Repository: Python-Activity-Based-Learning

INTRODUCTION

Activity-Based Learning helps students understand programming by applying concepts to practical problems. This project contains ten selected Python case studies that demonstrate problem-solving using Python programming.

The case studies cover arithmetic operations, conditional statements, loops, strings, lists, nested lists, tuples, and data analysis. Each case study includes a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshot, and conclusion.

OBJECTIVES OF THE PROJECT

1. To understand computational thinking and problem-solving.
2. To develop algorithms and pseudocode for programming problems.
3. To practise Python variables, data types, operators, and conditional statements.
4. To apply loops, strings, lists, nested lists, and tuples.
5. To execute Python programs and verify their outputs using test cases.
6. To organize the source code and output screenshots in a GitHub repository.

CASE STUDY 1: ATTENDANCE RECOVERY COMPUTATION

1. Problem Statement

Calculate a student's current attendance percentage and determine the minimum number of additional consecutive classes that must be attended to reach the required attendance percentage.

2. Objectives

• To calculate attendance percentage using arithmetic operations.

• To use conditional statements to check attendance eligibility.

• To use a loop to calculate the additional classes required.

3. Assumptions

• The number of attended and conducted classes is entered as a non-negative integer.

• Attended classes cannot exceed conducted classes.

• The target attendance percentage is between 0 and 100.

• Future classes are attended without missing any class.

4. Algorithm

1. Start the program.

2. Read the number of classes attended, classes conducted, and target percentage.

3. Validate the input values.

4. Calculate the current attendance percentage.

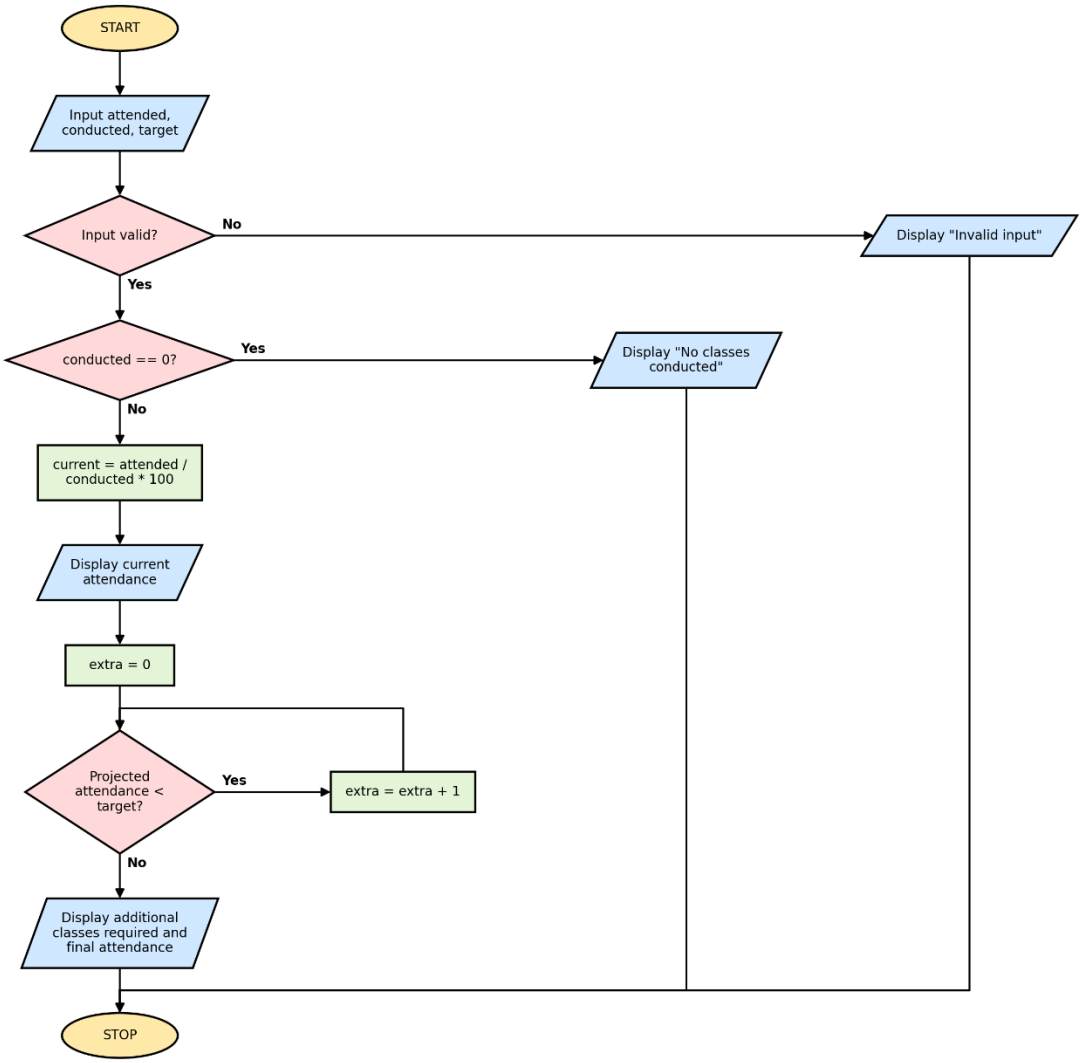
5. If the target is not reached, calculate the attendance percentage after each additional class.

6. Stop when the target percentage is reached.

7. Display the current attendance and additional classes required.

8. Stop the program.

5. Flowchart



6. Pseudocode

START

INPUT attended, conducted, target

IF input is invalid

DISPLAY error message

ELSE

CALCULATE current attendance percentage

SET extra classes to 0

WHILE target attendance is not reached

INCREASE extra classes by 1

UPDATE the projected attendance percentage

END WHILE

DISPLAY current attendance and extra classes

END IF

STOP

7. Python Source Code

```
attended = int(input("Enter classes attended: "))
conducted = int(input("Enter total classes conducted: "))
target = float(input("Enter required attendance percentage: "))
if attended < 0 or conducted < 0 or attended > conducted or target < 0 or target > 100:
 print("Invalid input")
elif conducted == 0:
 print("No classes conducted")
else:
 current = attended / conducted * 100
 print("Current attendance:", round(current, 2), "%")
 extra = 0
 while (attended + extra) / (conducted + extra) * 100 < target:
 extra += 1
 print("Additional classes required:", extra)
 print("Final attendance:",
 round((attended + extra) / (conducted + extra) * 100, 2), "%")
```

8. Test Cases

Test the program with:

• Attended classes: 60, conducted classes: 80, target: 75%.

• Attended classes equal to conducted classes.

• Invalid input where attended classes exceed conducted classes.

Record the expected and actual results according to your program's logic.

9. Output Screenshot



10. Conclusion

This program demonstrates how arithmetic operations, conditional statements, and loops can be used to calculate attendance and determine the additional classes required to reach a target percentage.

CASE STUDY 2: ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Develop a Python program to simulate ATM transactions such as deposits and withdrawals. The program maintains the account balance, rejects invalid transactions, and displays a transaction summary.

2. Objectives

• To understand transaction processing using Python.

• To use conditions to validate deposits and withdrawals.

• To use loops and collections to process multiple transactions.

3. Assumptions

• The opening balance is non-negative.

• Transaction amounts are represented in rupees.

• Deposits and withdrawals are processed according to the rules in the program.

• A withdrawal cannot exceed the available balance.

4. Algorithm

1. Start the program.

2. Set or read the opening account balance.

3. Read or access the transaction sequence.

4. Process each transaction.

5. Check whether the transaction type and amount are valid.

6. Add valid deposits to the balance.

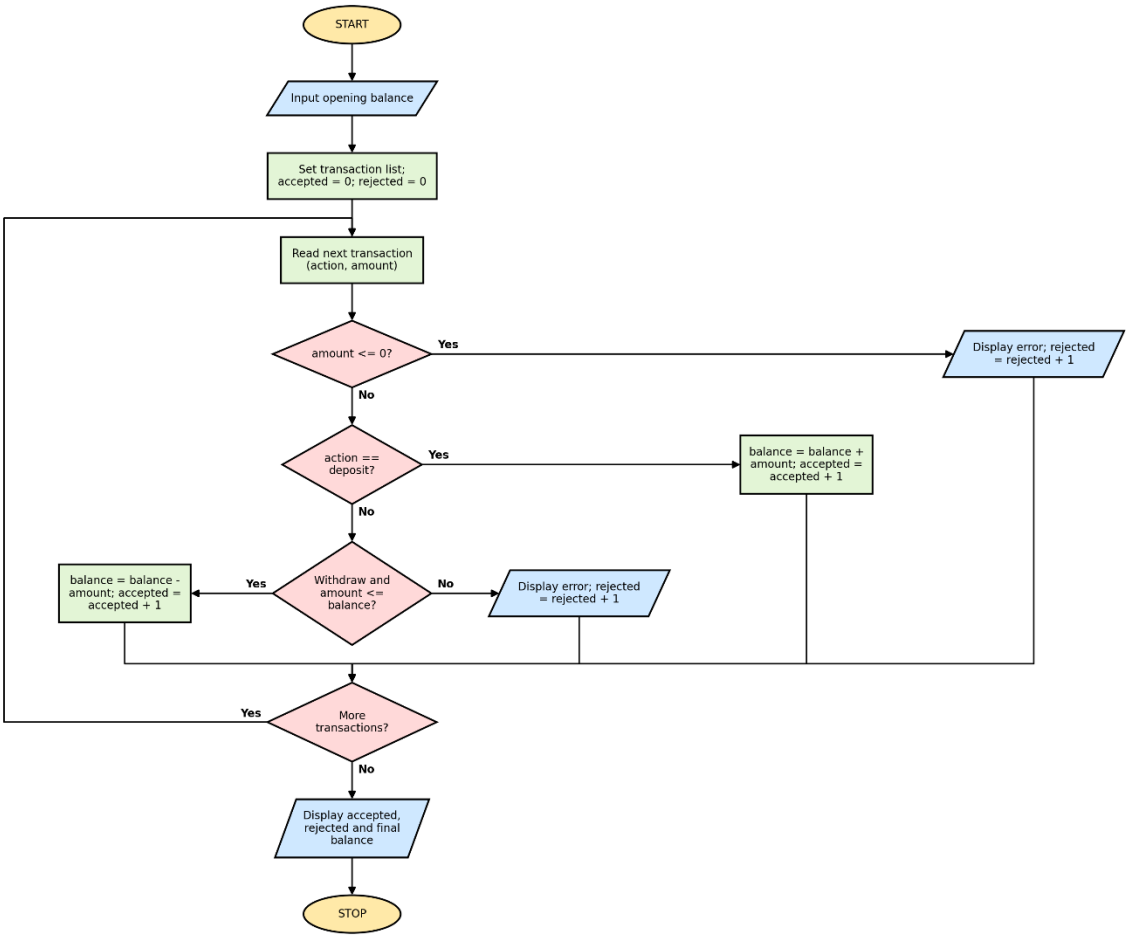
7. Subtract valid withdrawals when sufficient funds are available.

8. Reject invalid transactions.

9. Display the final balance and transaction summary.

10. Stop the program.

5. Flowchart



6. Pseudocode

START

SET opening balance

FOR each transaction

CHECK transaction type and amount

IF valid deposit

ADD amount to balance

ELSE IF valid withdrawal and sufficient balance

SUBTRACT amount from balance

ELSE

REJECT transaction

END IF

END FOR

DISPLAY final balance and transaction summary

STOP

7. Python Source Code

```
balance = float(input("Enter opening balance: "))
transactions = [
 ("deposit", 1500),
 ("withdraw", 400),
 ("withdraw", 5000),
 ("deposit", 750),
 ("withdraw", 250)
]
accepted = 0
rejected = 0
for action, amount in transactions:
 if amount <= 0:
 print("Invalid transaction amount")
 rejected += 1
 elif action == "deposit":
 balance += amount
 accepted += 1
 print("Deposited:", amount, "Balance:", balance)
 elif action == "withdraw":
 if amount <= balance:
 balance -= amount
 accepted += 1
 print("Withdrawn:", amount, "Balance:", balance)
```

```
else:
print("Insufficient balance for:", amount)
rejected += 1
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
print("Final balance:", balance)
```

8. Test Cases

• Deposit an amount into the account.

• Withdraw an amount smaller than the available balance.

• Attempt to withdraw more than the available balance.

Record the actual result produced by your program.

9. Output Screenshot



10. Conclusion

This program demonstrates the use of conditions, loops, and transaction data to simulate basic ATM operations and maintain an account balance.

CASE STUDY 3: STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Analyse student assessment marks across multiple examinations to calculate performance trends, identify improvement or decline, and summarize academic performance.

2. Objectives

• To store marks using Python lists or nested lists.

• To calculate averages and compare assessment results.

• To use loops and conditions for performance analysis.

3. Assumptions

• Marks are recorded according to the scale used in the program.

• The same subjects are compared across the assessments.

• Performance trends are determined from the marks stored in the program.

4. Algorithm

1. Start the program.

2. Store or read student assessment marks.

3. Process each student's marks or each subject's marks.

4. Calculate the required averages or differences.

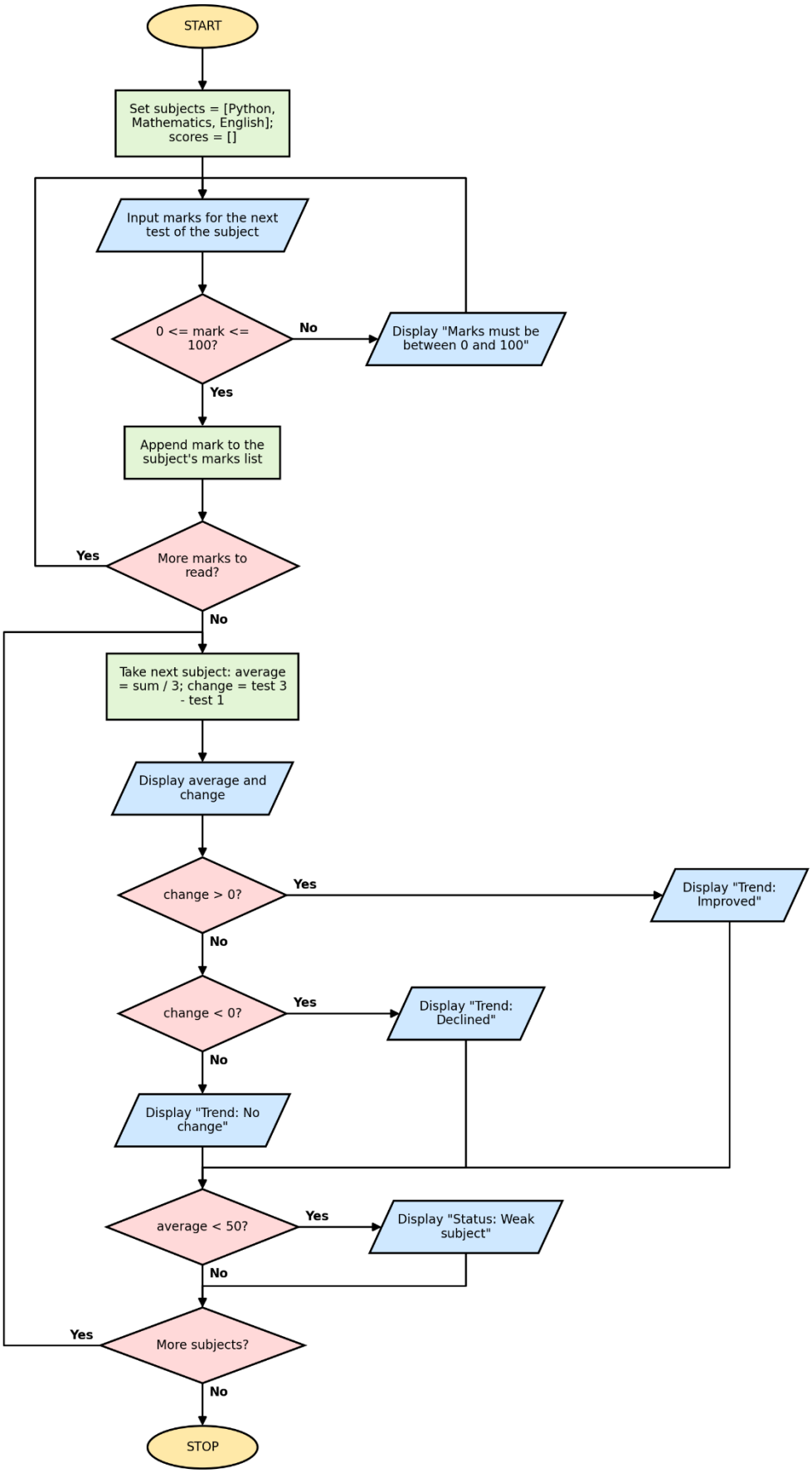
5. Compare the assessment results.

6. Identify improvement, decline, or consistent performance according to the program's rules.

7. Display the performance summary.

8. Stop the program.

5. Flowchart


START
Set subjects = [Python, Mathematics, English]; scores = []
Input marks for the next test of the subject
0 <= mark <= 100?
No
Display "Marks must be between 0 and 100"
Yes
Append mark to the subject's marks list
More marks to read?
Yes
No
Take next subject: average = sum / 3; change = test 3 - test 1
Display average and change
change > 0?
Yes
Display "Trend: Improved"
No
change < 0?
Yes
Display "Trend: Declined"
No
Display "Trend: No change"
average < 50?
Yes
Display "Status: Weak subject"
No
More subjects?
Yes
No
STOP

6. Pseudocode

START

READ or SET assessment marks

FOR each student or subject

CALCULATE required average or difference

COMPARE assessment results

CLASSIFY the trend using the program's conditions

END FOR

DISPLAY the performance summary

STOP

7. Python Source Code

```
subjects = ["Python", "Mathematics", "English"]
scores = []
for subject in subjects:
 marks = []
 for test in range(1, 4):
 mark = float(input(
 "Enter " + subject + " marks for test " + str(test) + ": "
 ))
 while mark < 0 or mark > 100:
 print("Marks must be between 0 and 100")
 mark = float(input("Enter marks again: "))
 marks.append(mark)
 scores.append(marks)
for i in range(len(subjects)):
 average = sum(scores[i]) / 3
 change = scores[i][2] - scores[i][0]
```

```
print("\nSubject:", subjects[i])

print("Average:", round(average, 2))

print("Change:", change)

if change > 0:

print("Trend: Improved")

elif change < 0:

print("Trend: Declined")

else:

print("Trend: No change")

if average < 50:

print("Status: Weak subject")
```

8. Test Cases

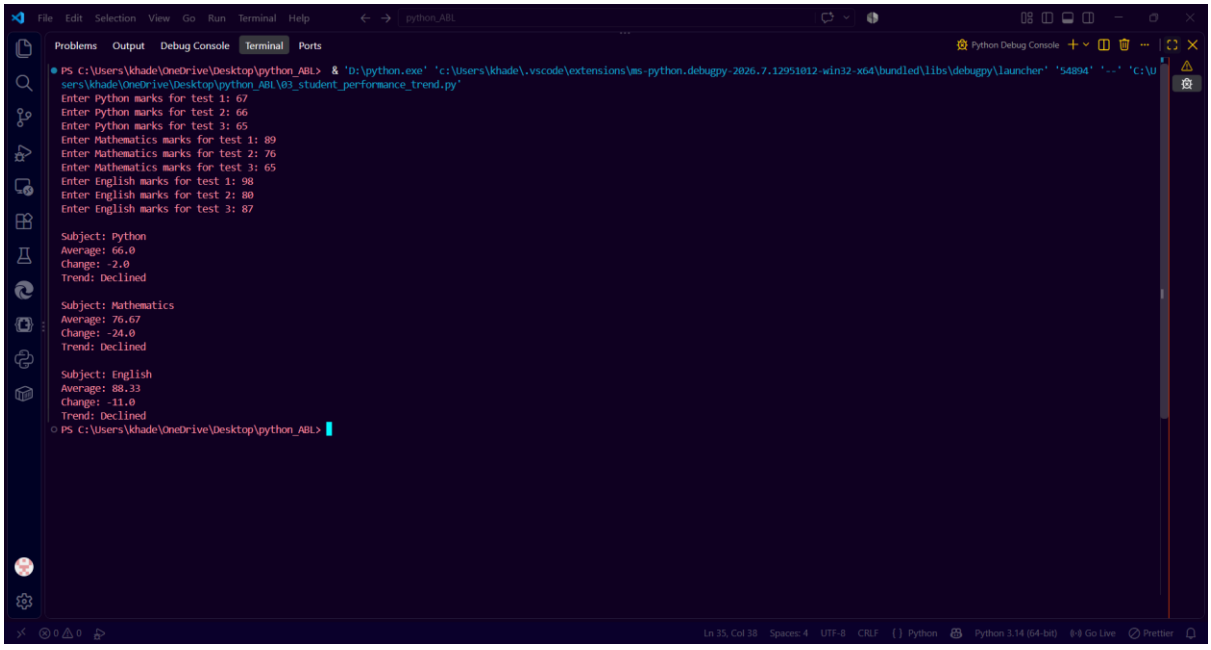
• A student whose marks increase across assessments.

• A student whose marks decrease across assessments.

• A student whose marks remain the same.

Use these scenarios only if they are supported by your program's input format.

9. Output Screenshot



10. Conclusion

This program demonstrates how lists, loops, arithmetic operations, and conditional statements can be used to analyse student performance trends.

CASE STUDY 4: REPEATED TRANSACTION DETECTION

1. Problem Statement

Develop a program to examine transaction records and identify repeated transaction values or records according to the rules used in the program.

2. Objectives

• To store transaction data in lists or tuples.

• To use loops to examine transaction records.

• To use membership checks or conditions to detect repeated entries.

3. Assumptions

• Transaction records are provided in the format expected by the program.

• A repeated transaction is identified using the comparison rules implemented in the code.

• The program analyses sample records and does not connect to a real banking system.

4. Algorithm

1. Start the program.

2. Read or initialize transaction records.

3. Examine the transaction records using a loop.

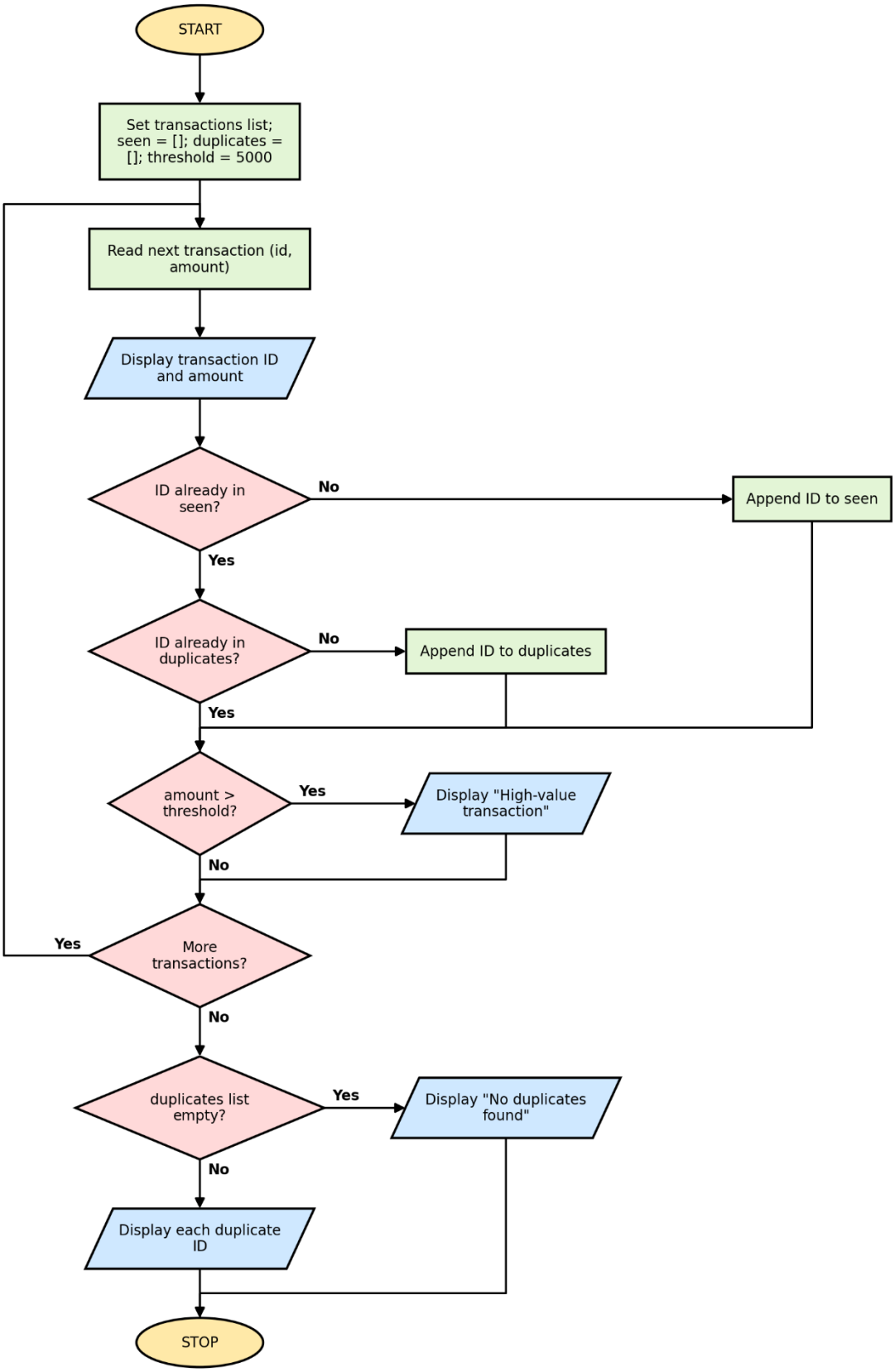
4. Compare records using the program's matching rule.

5. Identify repeated entries.

6. Display the detected repetitions or the appropriate message.

7. Stop the program.

5. Flowchart


START
Set transactions list; seen = []; duplicates = []; threshold = 5000
Read next transaction (id, amount)
Display transaction ID and amount
ID already in seen?
No
Append ID to seen
Yes
ID already in duplicates?
No
Append ID to duplicates
Yes
amount > threshold?
Yes
Display "High-value transaction"
No
More transactions?
Yes
No
duplicates list empty?
Yes
Display "No duplicates found"
No
Display each duplicate ID
STOP

6. Pseudocode

START

READ or SET transaction records

CHECK records using the required comparison rule

FOR each record

DETERMINE whether a matching record already exists

IF repeated

RECORD the repeated entry

END IF

END FOR

DISPLAY the result

STOP

7. Python Source Code

```
transactions = [
 ("TX101", 450),
 ("TX102", 1200),
 ("TX103", 850),
 ("TX102", 1200),
 ("TX104", 6200),
 ("TX105", 300)
]
seen = []
duplicates = []
threshold = 5000
print("Transaction Details")
for transaction_id, amount in transactions:
 print(transaction_id, "Amount:", amount)
 if transaction_id in seen:
```

```
if transaction_id not in duplicates:
duplicates.append(transaction_id)
else:
seen.append(transaction_id)
if amount > threshold:
print("High-value transaction:", transaction_id)
print("\nDuplicate transaction IDs:")
if duplicates:
for transaction_id in duplicates:
print(transaction_id)
else:
print("No duplicates found")
```

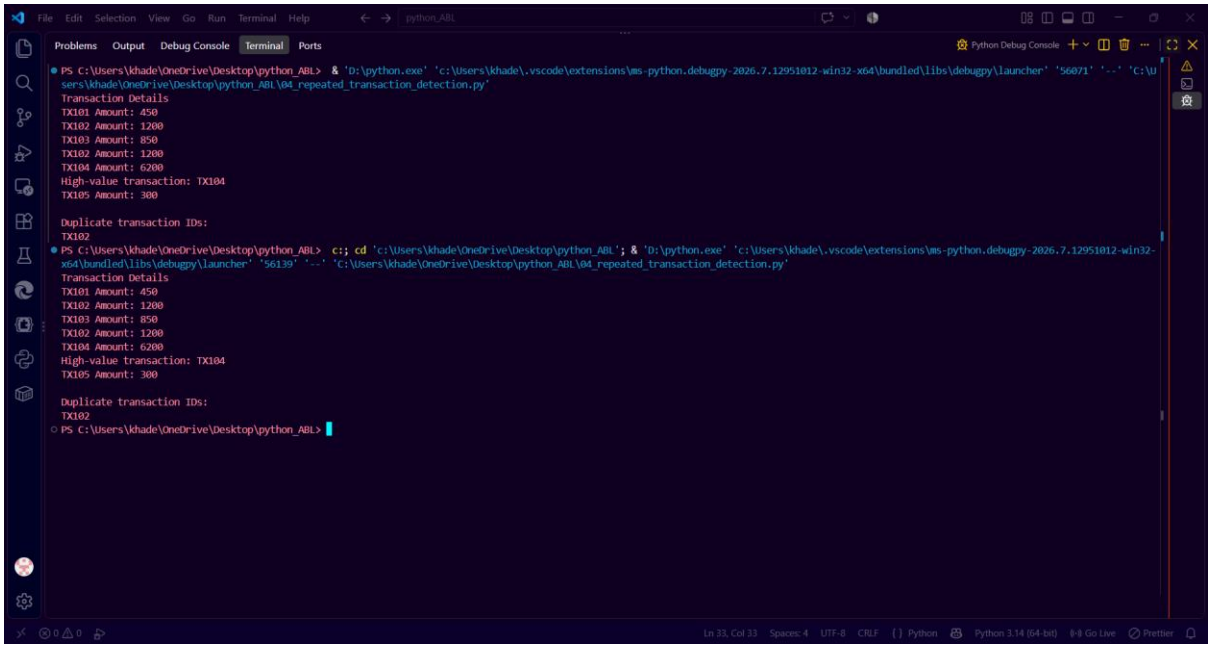
8. Test Cases

• A set of transactions containing a repeated entry.

• A set of transactions containing no repeated entries.

• A set containing multiple repeated entries, if supported by the program.

9. Output Screenshot



10. Conclusion

This program demonstrates how Python collections, loops, and membership checks can be used to identify repeated transaction records in sample data.

CASE STUDY 5: STRING-BASED EXAMINATION ANSWER EVALUATION

1. Problem Statement

Create a Python program that compares student answer strings with the expected answers and evaluates the results according to the comparison rules in the program.

2. Objectives

• To practise Python string operations.

• To compare answer strings using conditions.

• To count correct answers or calculate a score when supported by the program.

3. Assumptions

• Student answers and expected answers are available in the required format.

• Answer comparison follows the rules implemented in the code.

• The program is a basic demonstration and does not understand the meaning of written answers.

4. Algorithm

1. Start the program.

2. Store or read the expected answers and student answers.

3. Process the answers one by one.

4. Compare each student answer with its expected answer.

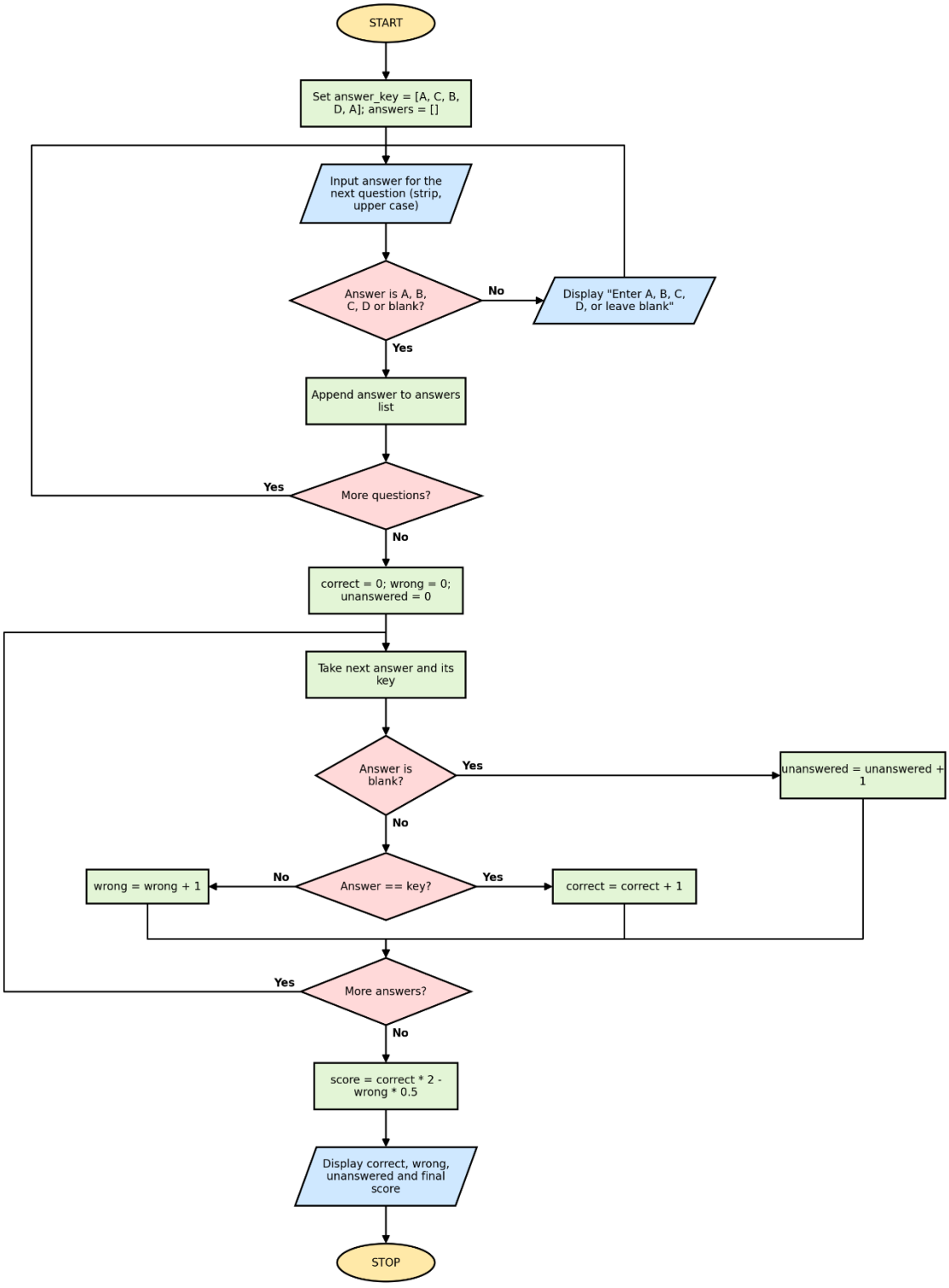
5. Record the result of each comparison.

6. Calculate the total score if included in the program.

7. Display the evaluation summary.

8. Stop the program.

5. Flowchart



6. Pseudocode

START

READ or SET expected answers and student answers

SET score to the initial value used by the program

FOR each answer pair

COMPARE the student answer with the expected answer

IF the comparison is correct

UPDATE the result or score

END IF

END FOR

DISPLAY evaluation results

STOP

7. Python Source Code

```
answer_key = ["A", "C", "B", "D", "A"]
answers = []
for i in range(len(answer_key)):
 answer = input(
 "Enter answer for question " + str(i + 1) + ": "
 ).strip().upper()
 while answer not in ["A", "B", "C", "D", ""]:
 print("Enter A, B, C, D, or leave blank")
 answer = input("Enter answer again: ").strip().upper()
 answers.append(answer)
correct = 0
wrong = 0
unanswered = 0
for i in range(len(answer_key)):
 if answers[i] == "":
 unanswered += 1
 elif answers[i] == answer_key[i]:
 correct += 1
```

```
else:
wrong += 1
score = correct * 2 - wrong * 0.5
print("\nExamination Result")
print("Correct answers:", correct)
print("Wrong answers:", wrong)
print("Unanswered:", unanswered)
print("Final score:", score, "out of", len(answer_key) * 2)
```

8. Test Cases

• Answers that match the expected answers.

• Answers that do not match.

• A mixture of matching and non-matching answers.

9. Output Screenshot



10. Conclusion

This program demonstrates how strings, comparisons, loops, and counters can be used to evaluate answers in a simple examination system.

CASE STUDY 6: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Use a nested list or matrix to store student marks and calculate the required row-wise, column-wise, or overall summaries according to the program's design.

2. Objectives

• To understand nested lists in Python.

• To access matrix elements using row and column indexes.

• To use nested loops for marks analysis.

3. Assumptions

• The matrix contains marks in the format expected by the program.

• Each row and column represents the student or subject specified by the program.

• The program calculates only the summaries included in its implementation.

4. Algorithm

1. Start the program.

2. Create or read the marks matrix.

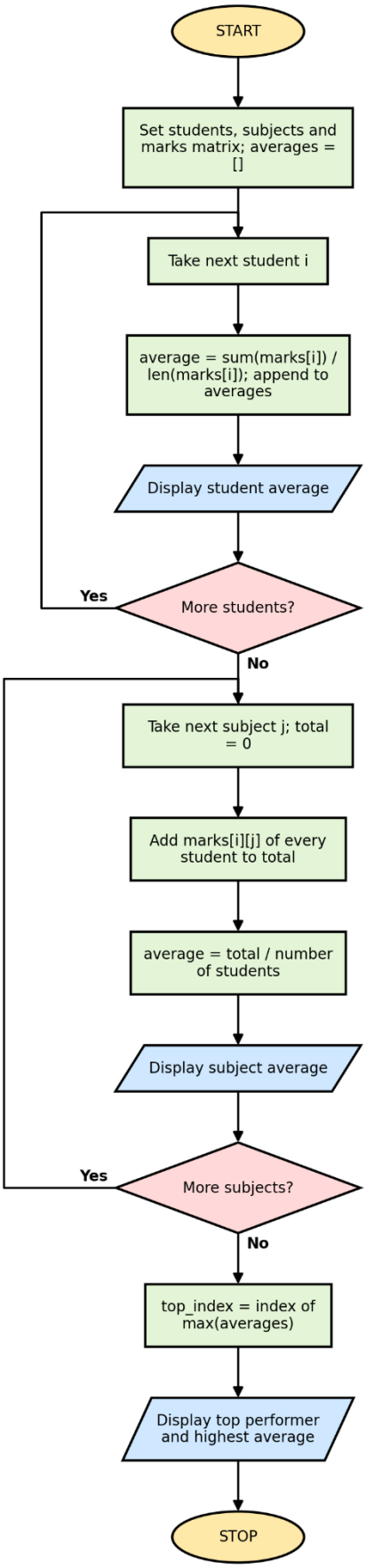
3. Use loops to access the rows and columns.

4. Calculate the required totals or summaries.

5. Display the results.

6. Stop the program.

5. Flowchart


START
Set students, subjects and marks matrix; averages = []
Take next student i
average = sum(marks[i]) / len(marks[i]); append to averages
Display student average
More students?
Yes
No
Take next subject j; total = 0
Add marks[i][j] of every student to total
average = total / number of students
Display subject average
More subjects?
Yes
No
top_index = index of max(averages)
Display top performer and highest average
STOP

6. Pseudocode

START

READ or SET marks matrix

FOR each row

PROCESS the marks in that row

CALCULATE the required row summary

END FOR

PROCESS columns if required

DISPLAY the matrix analysis

STOP

7. Python Source Code

```
students = ["Asha", "Ravi", "Noor", "Kiran"]
subjects = ["Python", "Maths", "English"]
marks = [
 [88, 76, 91],
 [67, 82, 73],
 [94, 89, 96],
 [55, 61, 58]
]
print("Student Averages")
averages = []
for i in range(len(students)):
 average = sum(marks[i]) / len(marks[i])
 averages.append(average)
 print(students[i], ":", round(average, 2))
print("\nSubject Averages")
for j in range(len(subjects)):
```

```
total = 0
for i in range(len(students)):
total += marks[i][j]
average = total / len(students)
print(subjects[j], ":", round(average, 2))
top_index = averages.index(max(averages))
print("\nTop Performer:", students[top_index])
print("Highest Average:", round(averages[top_index], 2))
```

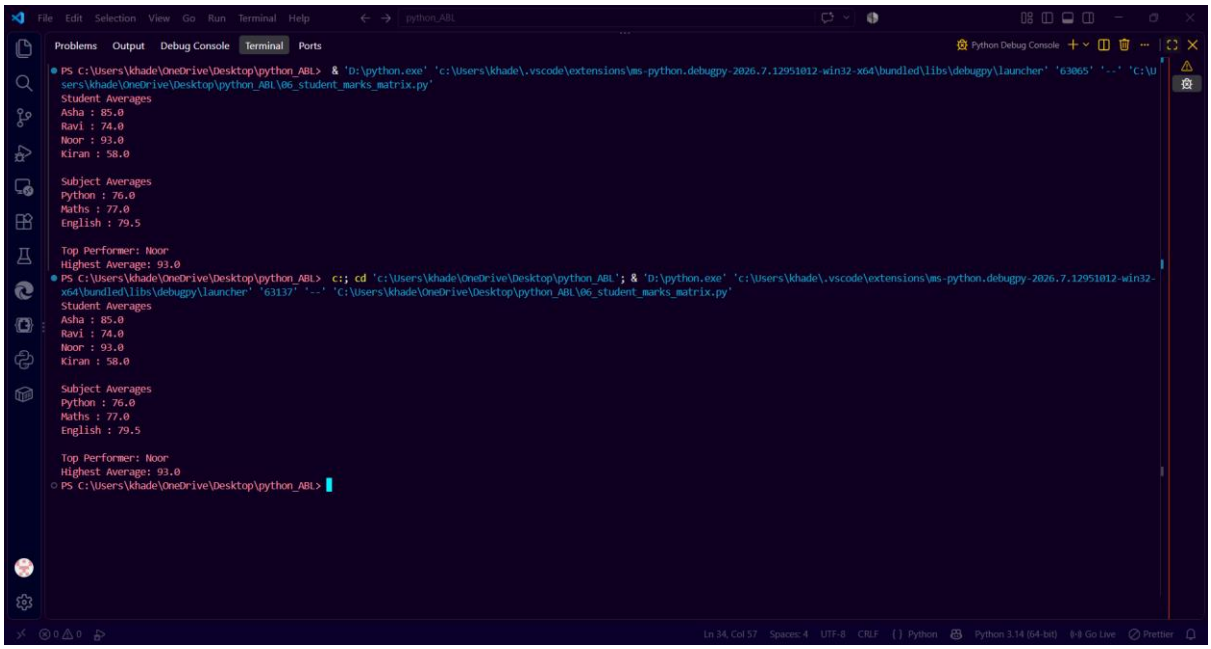
8. Test Cases

• A matrix containing multiple students and subjects.

• A matrix containing different marks values.

• A small matrix to verify totals manually.

9. Output Screenshot



10. Conclusion

This program demonstrates how nested lists and nested loops can be used to organize and analyse marks in a matrix format.

CASE STUDY 7: WEEKLY ELECTRICITY CONSUMPTION MATRIX

1. Problem Statement

Represent electricity consumption using a matrix and process the recorded values to calculate the required daily or weekly consumption summaries.

2. Objectives

• To store consumption data using nested lists.

• To use nested loops to process rows and columns.

• To calculate totals or comparisons according to the program's rules.

3. Assumptions

• Consumption values are provided in the format expected by the program.

• Each row and column represents a time period or location as defined in the code.

• All calculations follow the units and logic used in the program.

4. Algorithm

1. Start the program.

2. Create or read the electricity consumption matrix.

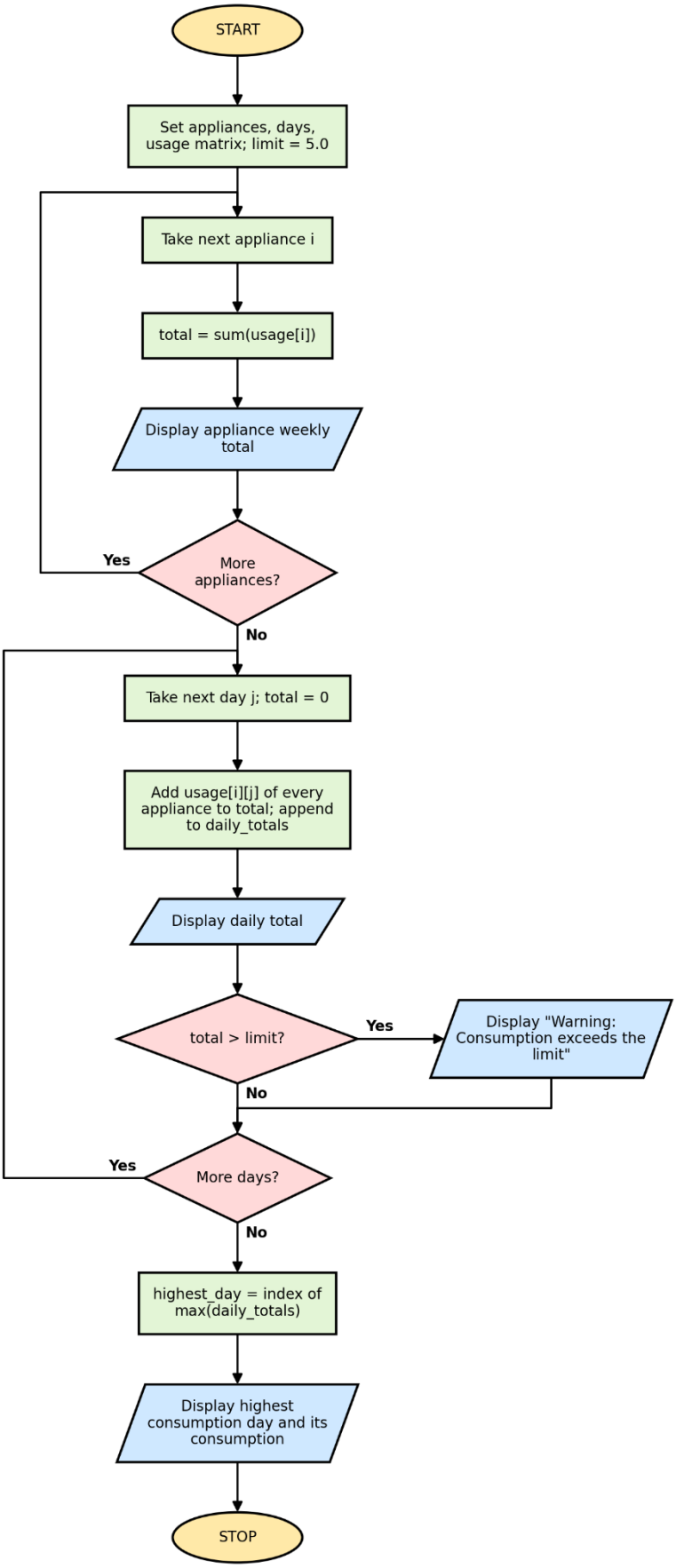
3. Traverse the matrix using nested loops.

4. Calculate the required totals or comparisons.

5. Display the consumption analysis.

6. Stop the program.

5. Flowchart


START
Set appliances, days, usage matrix; limit = 5.0
Take next appliance i
total = sum(usage[i])
Display appliance weekly total
More appliances?
Yes
No
Take next day j; total = 0
Add usage[i][j] of every appliance to total; append to daily_totals
Display daily total
total > limit?
Yes
Display "Warning: Consumption exceeds the limit"
No
More days?
Yes
No
highest_day = index of max(daily_totals)
Display highest consumption day and its consumption
STOP

6. Pseudocode

START

READ or SET consumption matrix

FOR each row

SET the required total to its initial value

FOR each consumption value in the row

ADD or process the value as required

END FOR

STORE or DISPLAY the row result

END FOR

DISPLAY the final analysis

STOP

7. Python Source Code

```
appliances = ["Fan", "Light", "Computer"]
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
usage = [
 [1.2, 1.1, 1.3, 1.2, 1.4, 1.6, 1.5],
 [0.5, 0.5, 0.4, 0.5, 0.6, 0.7, 0.6],
 [2.0, 2.5, 2.2, 2.8, 3.0, 3.5, 3.1]
]
limit = 5.0
print("Weekly Appliance Consumption")
for i in range(len(appliances)):
 total = sum(usage[i])
 print(appliances[i], ":", round(total, 2), "kWh")
print("\nDaily Consumption")
daily_totals = []
```

```
for j in range(len(days)):
 total = 0
 for i in range(len(appliances)):
 total += usage[i][j]
 daily_totals.append(total)
 print(days[j], ":", round(total, 2), "kWh")
 if total > limit:
 print("Warning: Consumption exceeds the limit")
highest_day = daily_totals.index(max(daily_totals))
print("\nHighest Consumption Day:", days[highest_day])
print("Consumption:", round(daily_totals[highest_day], 2), "kWh")
```

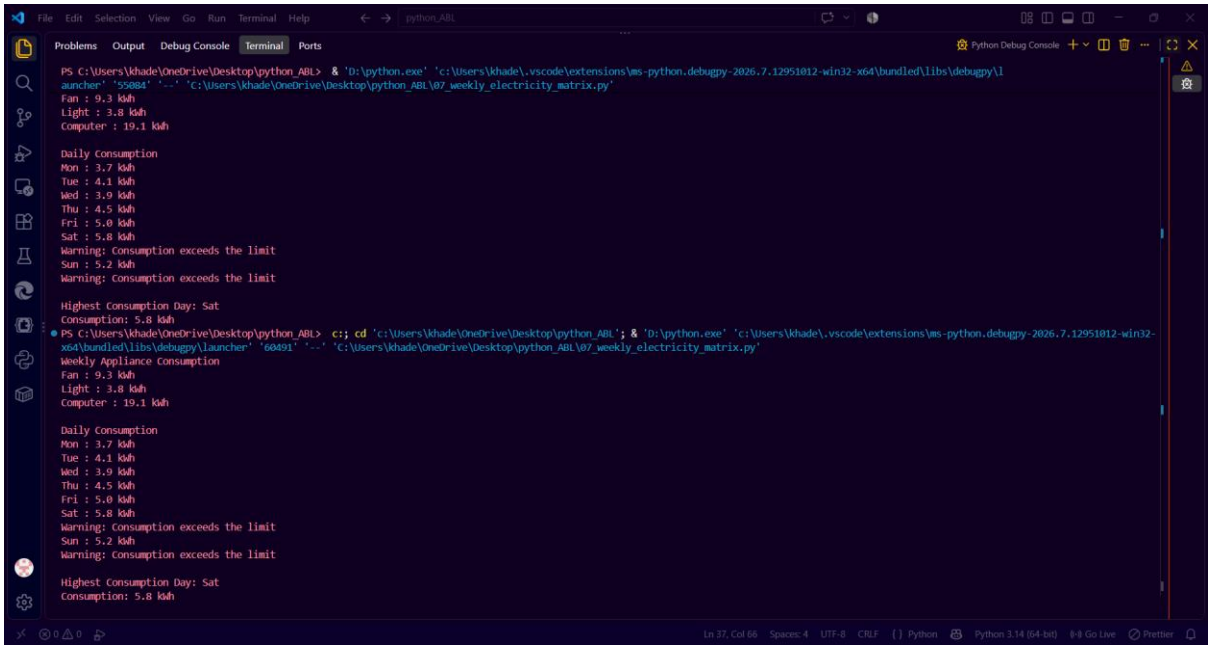
8. Test Cases

• A matrix with different consumption values.

• A row with all equal values.

• A small matrix whose total can be checked manually.

9. Output Screenshot



10. Conclusion

This program demonstrates how nested lists and nested loops can be used to organize

and analyse electricity consumption data.

CASE STUDY 8: TUPLE-BASED PRODUCT CATALOGUE AUDIT

1. Problem Statement

Use tuples to store product information and examine the catalogue to produce the audit or summary required by the program.

2. Objectives

• To understand tuple creation and access.

• To practise tuple unpacking.

• To use conditions and loops to examine product records.

3. Assumptions

• Product information is stored in the tuple format expected by the program.

• Each tuple field has the meaning assigned to it in the source code.

• The audit follows the conditions implemented in the program.

4. Algorithm

1. Start the program.

2. Create or read the product catalogue.

3. Visit each product record.

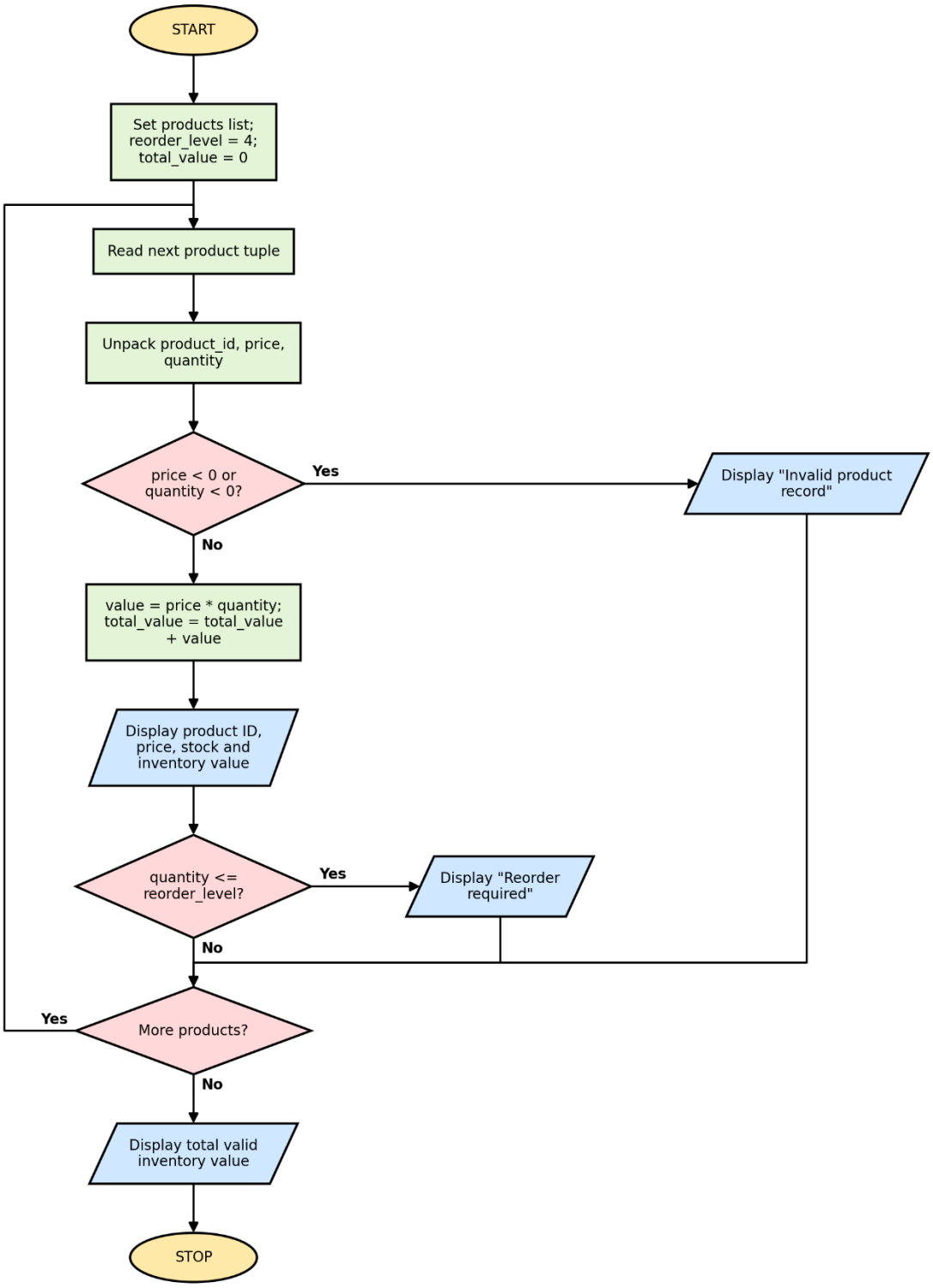
4. Unpack or access the required tuple values.

5. Check each record using the program's conditions.

6. Display the audit results.

7. Stop the program.

5. Flowchart



6. Pseudocode

START

READ or SET product catalogue

FOR each product tuple

ACCESS or UNPACK the product fields

CHECK the record using the specified conditions

RECORD the audit result

END FOR

DISPLAY the catalogue audit

STOP

7. Python Source Code

```
products = [
 ("P101", 499, 12),
 ("P102", 0, 5),
 ("P103", 1299, 0),
 ("P104", 249.50, 3),
 ("P105", 799, -2)
]
reorder_level = 4
total_value = 0
print("Product Catalogue Report")
for product_id, price, quantity in products:
 if price < 0 or quantity < 0:
 print(product_id, ": Invalid product record")
 continue
 value = price * quantity
 total_value += value
 print("\nProduct ID:", product_id)
 print("Price:", price)
 print("Stock:", quantity)
```

```
print("Inventory Value:", value)
if quantity <= reorder_level:
print("Reorder required")
print("\nTotal Valid Inventory Value:", total_value)
```

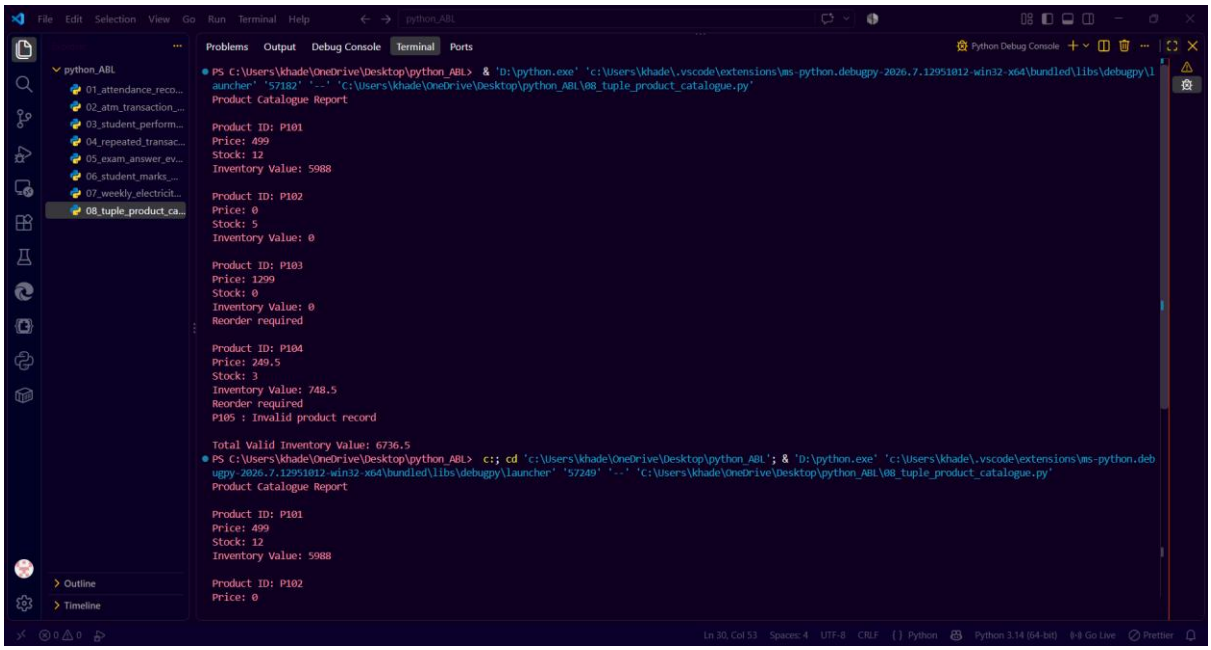
8. Test Cases

• A catalogue containing records that satisfy the program's rules.

• A record that fails one of the implemented checks.

• A catalogue containing multiple product records.

9. Output Screenshot



10. Conclusion

This program demonstrates the use of tuples, tuple unpacking, loops, and conditions to process structured product information.

CASE STUDY 9: STUDENT ACADEMIC RISK CLASSIFICATION

1. Problem Statement

Develop a program that examines student academic information and classifies records according to the risk conditions defined in the source code.

2. Objectives

• To store student records using Python collections.

• To use conditions to classify academic records.

• To practise loops and logical expressions.

3. Assumptions

• Student information is available in the format expected by the program.

• Risk categories and their conditions are defined by the program.

• The classification is a demonstration based on the supplied sample data.

4. Algorithm

1. Start the program.

2. Read or initialize the student records.

3. Examine each student's academic information.

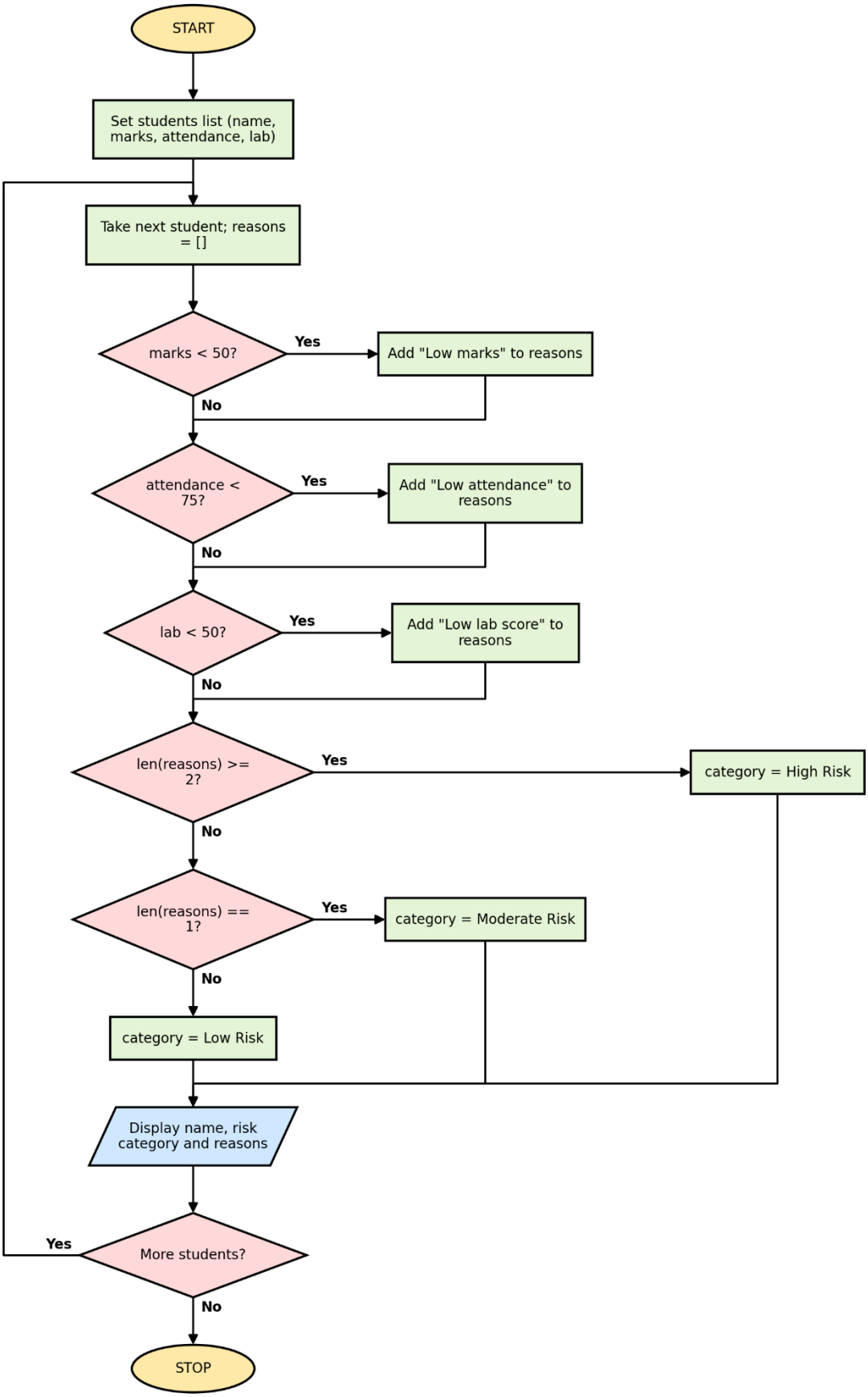
4. Apply the risk conditions defined in the code.

5. Assign the relevant classification.

6. Display the classification results.

7. Stop the program.

5. Flowchart


START
Set students list (name, marks, attendance, lab)
Take next student; reasons = []
marks < 50?
Yes
Add "Low marks" to reasons
No
attendance < 75?
Yes
Add "Low attendance" to reasons
No
lab < 50?
Yes
Add "Low lab score" to reasons
No
len(reasons) >= 2?
Yes
category = High Risk
No
len(reasons) == 1?
Yes
category = Moderate Risk
No
category = Low Risk
Display name, risk category and reasons
More students?
Yes
No
STOP

6. Pseudocode

START

READ or SET student records

FOR each student record

ACCESS the required academic information

CHECK the risk conditions

ASSIGN the classification according to the conditions

END FOR

DISPLAY the classification results

STOP

7. Python Source Code

```
students = [
 ("Asha", 78, 92, 85),
 ("Ravi", 43, 68, 55),
 ("Noor", 61, 74, 42),
 ("Kiran", 35, 52, 30)
]
print("Student Academic Risk Report")
for name, marks, attendance, lab in students:
 reasons = []
 if marks < 50:
 reasons.append("Low marks")
 if attendance < 75:
 reasons.append("Low attendance")
 if lab < 50:
 reasons.append("Low lab score")
 if len(reasons) >= 2:
```

```
category = "High Risk"
elif len(reasons) == 1:
category = "Moderate Risk"
else:
category = "Low Risk"
print("\nStudent:", name)
print("Risk Category:", category)
if reasons:
print("Reasons:", ", ".join(reasons))
else:
print("Reasons: None")
```

8. Test Cases

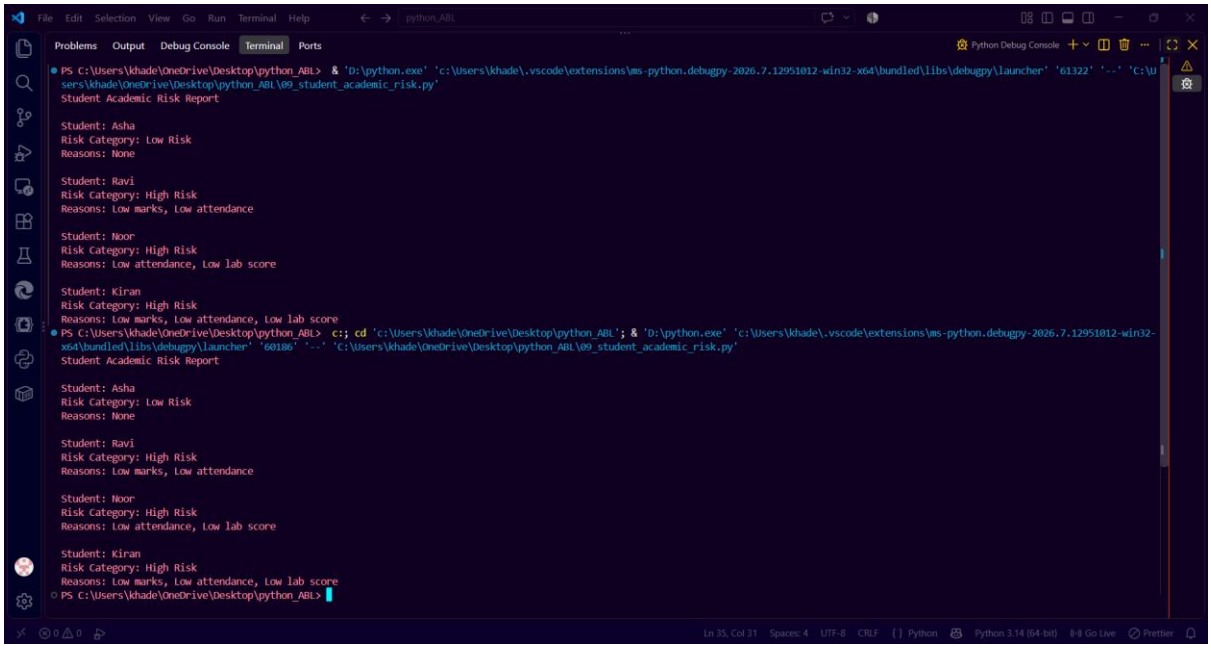
• A student record that satisfies the program's low-risk conditions.

• A student record that satisfies the program's higher-risk conditions.

• A record near a classification boundary.

Use these scenarios only if they match the actual conditions in your code.

9. Output Screenshot



10. Conclusion

This program demonstrates how Python collections, logical expressions, and conditional statements can be used to classify student records according to specified rules.

CASE STUDY 10: SMART ELECTRICITY BILLING FOR MULTIPLE APARTMENTS

1. Problem Statement

Create a Python program to process electricity consumption for multiple apartments and calculate bills according to the billing rules implemented in the program.

2. Objectives

• To calculate electricity charges using Python arithmetic.

• To apply conditions or functions for billing.

• To process multiple apartment records using collections and loops.

3. Assumptions

• Consumption values are entered or stored in the format expected by the program.

• The billing rates and conditions are those specified in the source code.

• The program calculates sample bills and is not connected to a real electricity provider.

4. Algorithm

1. Start the program.

2. Read or initialize apartment records and consumption values.

3. Process each apartment record.

4. Calculate charges using the implemented billing rules.

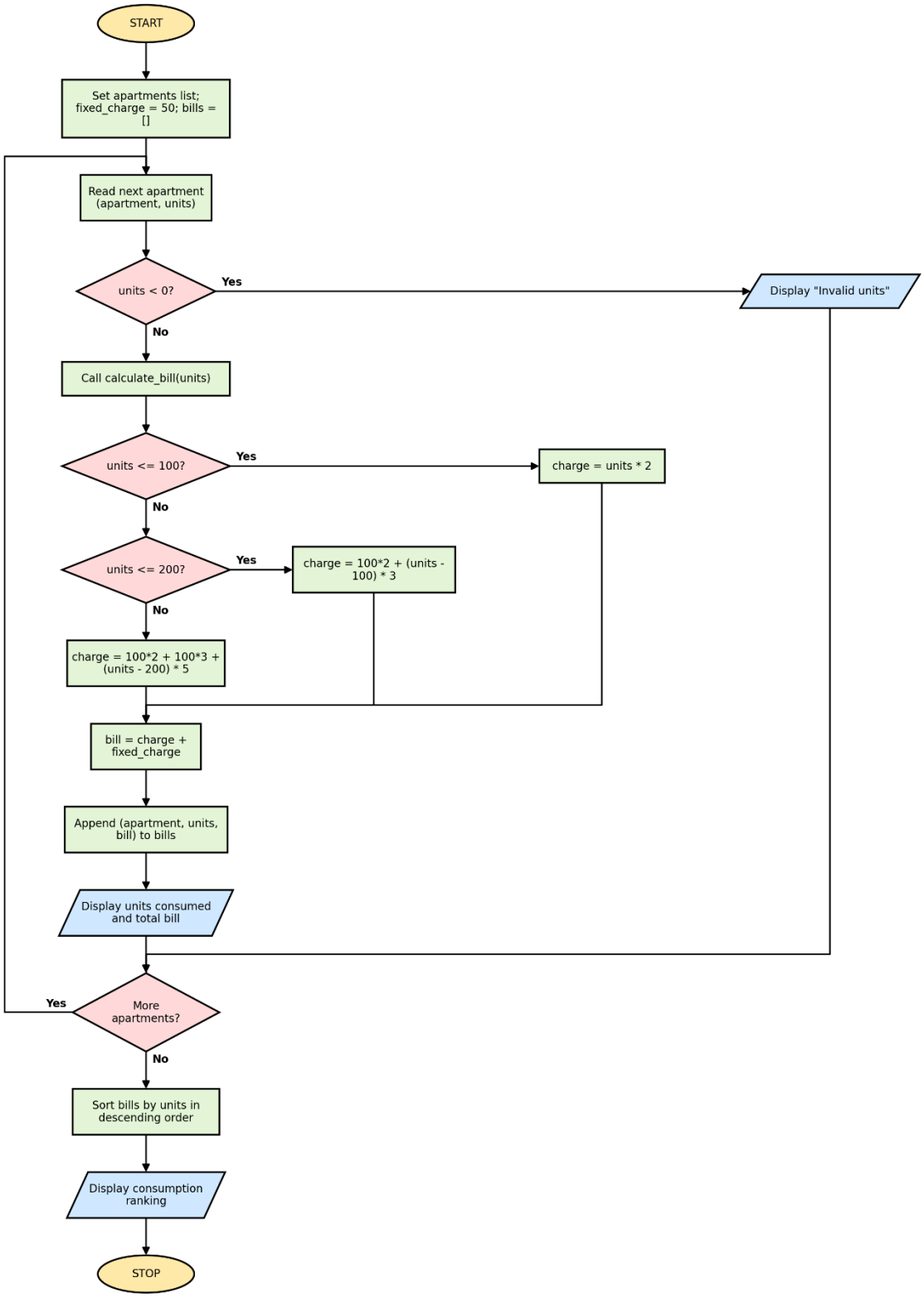
5. Apply any conditions or additional charges specified in the code.

6. Store or display the bill for each apartment.

7. Display the final billing summary.

8. Stop the program.

5. Flowchart



6. Pseudocode

START

READ or SET apartment records

FOR each apartment

GET the consumption value

CALCULATE the bill using the program's billing rules

APPLY any required conditions

STORE the calculated bill

END FOR

DISPLAY the apartment billing summary

STOP

7. Python Source Code

```
apartments = [
 ("A-101", 85),
 ("A-102", 145),
 ("A-103", 230),
 ("A-104", 62),
 ("A-105", 190)
]
fixed_charge = 50
def calculate_bill(units):
 if units <= 100:
 charge = units * 2
 elif units <= 200:
 charge = 100 * 2 + (units - 100) * 3
 else:
 charge = 100 * 2 + 100 * 3 + (units - 200) * 5
 return charge + fixed_charge
bills = []
print("Apartment Electricity Bills")
```

```
for apartment, units in apartments:
 if units < 0:
 print(apartment, ": Invalid units")
 continue
 bill = calculate_bill(units)
 bills.append((apartment, units, bill))
 print("\nApartment:", apartment)
 print("Units Consumed:", units)
 print("Total Bill: Rs.", bill)
bills.sort(key=lambda item: item[1], reverse=True)
print("\nConsumption Ranking")
for rank, (apartment, units, bill) in enumerate(bills, start=1):
 print(rank, apartment, "-", units, "units, Bill: Rs.", bill)
```

8. Test Cases

• An apartment with low electricity consumption.

• An apartment with higher electricity consumption.

• Multiple apartments with different consumption values.

Record the expected and actual bills using the rates and rules in your actual program.

9. Output Screenshot



10. Conclusion

This program demonstrates how Python arithmetic, functions, conditions, and collections can be used to process electricity bills for multiple apartments.